Note de séance SQL n°8

Modélisation : types, contraintes, clés, normalisation | 2026-10-24

1. Les notions

Notion	Règle à retenir
Types	INT (entiers), DECIMAL(p,s) (montants exacts), VARCHAR(n) (texte), DATE / DATETIME. Un téléphone ou u
Contraintes	PRIMARY KEY (unique, non NULL), AUTO_INCREMENT, NOT NULL, DEFAULT, UNIQUE, CHECK (appliqu
Codes d'erreur	1048 : NULL dans une colonne NOT NULL. 1292 : date invalide. 1366 : valeur de mauvais type. 3819 : CHEC
Clé étrangère	Renvoie vers une clé primaire (ou UNIQUE). Même type des deux côtés, toutes les valeurs existantes doivent
ON DELETE	RESTRICT (défaut, refuse), CASCADE (supprime les enfants), SET NULL. En pratique, on annule par un stat
Formes normales	1FN : une valeur par cellule. 2FN : chaque colonne dépend de toute la clé. 3FN : chaque colonne dépend de l
Anomalies	Mise à jour (données contradictoires), insertion (inventer des données pour en saisir d'autres), suppression (p
Bonnes pratiques	Ne jamais tester sur les vraies données : travailler sur une copie. Protéger un DELETE par START TRANSAC

2. Partie 1 : types et contraintes

1 : Table livraisons

```
CREATE TABLE livraisons (
    id INT AUTO_INCREMENT PRIMARY KEY,
    commande_id INT NOT NULL,
    date_livraison DATE,
    statut VARCHAR(20) NOT NULL DEFAULT 'en préparation',
    cout DECIMAL(8,2) NOT NULL CHECK (cout >= 0),
    transporteur VARCHAR(50)
);
```

Pièges : défaut mal orthographié, CHECK strict (>) au lieu de >=.

2 : Trois livraisons (la troisième utilise les valeurs par défaut)

```
INSERT INTO livraisons (commande_id, date_livraison, statut, cout, transporteur)
VALUES (1, '2026-01-16', 'livrée', 12.50, 'DHL');
INSERT INTO livraisons (commande_id, date_livraison, statut, cout, transporteur)
VALUES (3, '2026-02-12', 'livrée', 40.00, 'DHL');
INSERT INTO livraisons (commande_id, cout) VALUES (6, 15.75);
```

La troisième ligne : statut 'en préparation', date et transporteur NULL.

3 : Quatre insertions invalides

```
-- (a) cout NULL            -> erreur 1048
-- (b) date '2026-13-45'    -> erreur 1292
-- (c) cout 'abc'           -> erreur 1366
-- (d) cout -5              -> erreur 3819 (CHECK)
```

4 : Téléphone en INT

```
CREATE TEMPORARY TABLE test_tel (tel INT);
INSERT INTO test_tel VALUES ('0812345678');
SELECT * FROM test_tel;
```

Résultat : 812345678. Le zéro initial est perdu ; un INT n'accepte ni '+' ni espaces et déborde sur les longs numéros.

5 : Précision des nombres

```
SELECT 0.1 + 0.2 = 0.3;           -- 1 (DECIMAL, exact)
SELECT 0.1e0 + 0.2e0 = 0.3e0;     -- 0 (DOUBLE, approché)
```

FLOAT et DOUBLE stockent en binaire, où 0,1 n'a pas de représentation exacte : à éviter pour les montants.

7 : Modèle recommandé pour la table commandes

```
-- id            INT AUTO_INCREMENT PRIMARY KEY
-- client_id     INT NOT NULL      (clé étrangère vers clients, bloquée par la commande orpheline)
-- montant       DECIMAL(10,2) NOT NULL CHECK (montant >= 0)
-- date_commande DATE NOT NULL     (ne pas nommer une colonne date)
-- produit_id    INT NOT NULL      (clé étrangère vers produits)
-- produit       texte redondant : à supprimer (dépendance transitive, 3FN)
```

3. Partie 2 : clés primaires et étrangères

1 : Clés primaires

```
ALTER TABLE clients ADD PRIMARY KEY (id);
ALTER TABLE commandes ADD PRIMARY KEY (id);
ALTER TABLE villes ADD PRIMARY KEY (ville);
```

2 : Clés étrangères

```
ALTER TABLE commandes ADD CONSTRAINT fk_commandes_produit
    FOREIGN KEY (produit_id) REFERENCES produits(id);
ALTER TABLE clients ADD CONSTRAINT fk_clients_ville
    FOREIGN KEY (ville) REFERENCES villes(ville);
ALTER TABLE livraisons ADD CONSTRAINT fk_livraisons_commande
    FOREIGN KEY (commande_id) REFERENCES commandes(id);
```

3 : Tests d'intégrité

```
INSERT INTO livraisons (commande_id, date_livraison, statut, cout, transporteur)
VALUES (99, '2026-12-01', 'livrée', 12.50, 'DHL');      -- erreur 1452

START TRANSACTION;
DELETE FROM commandes WHERE id = 1;                      -- erreur 1451
ROLLBACK;
SELECT COUNT(*) FROM commandes;                          -- 7
```

4 : La commande orpheline, sur des copies

```
CREATE TABLE commandes_copie LIKE commandes;
INSERT INTO commandes_copie SELECT * FROM commandes;
CREATE TABLE clients_copie LIKE clients;
INSERT INTO clients_copie SELECT * FROM clients;

-- La clé étrangère échoue (erreur 1452) : le client 9 n'existe pas.
-- Repérer la ligne fautive :
SELECT o.*
FROM commandes o
LEFT JOIN clients c ON o.client_id = c.id
WHERE c.id IS NULL;

-- Solution retenue : un client « inconnu » qui garde l'id 9, sans attribut inventé
INSERT INTO clients_copie (id, prenom) VALUES (9, 'Client inconnu');
ALTER TABLE commandes_copie ADD CONSTRAINT fk_commandes_copie_clients
    FOREIGN KEY (client_id) REFERENCES clients_copie(id);
```

Supprimer la commande perd 120 de chiffre d'affaires ; NULL perd l'identifiant 9 ; le client inconnu garde la vente et rend l'anomalie visible. La vraie correction est de retrouver le client 9 à la source.

4. Partie 3 : normalisation

1-3 : Anomalies de la table à plat ventes_plat

```
SELECT ville_client, province_client, COUNT(*) AS nb_repetitions
FROM ventes_plat
```

```
GROUP BY ville_client, province_client;          -- Kinshasa 5, Lubumbashi 1

UPDATE ventes_plat SET province_client = 'Kinshasa-Ville' WHERE id_vente = 1;
SELECT DISTINCT ville_client, province_client FROM ventes_plat ORDER BY 1, 2;   -- 3 lignes
UPDATE ventes_plat SET province_client = 'Kinshasa' WHERE id_vente = 1;         -- remise en ordre
```

Insertion : impossible d'enregistrer un produit sans vente (il faudrait inventer une vente fictive). Suppression de la vente 3 : on perd le seul client de Lubumbashi, la province Haut-Katanga et le produit Ordinateur.

4 : Schéma normalisé

```
CREATE TABLE villes_n (
    ville VARCHAR(50) PRIMARY KEY,
    province VARCHAR(50) NOT NULL
);
CREATE TABLE clients_n (
    id INT AUTO_INCREMENT PRIMARY KEY,
    prenom VARCHAR(50) NOT NULL,
    ville VARCHAR(50) NOT NULL,
    CONSTRAINT fk_clients_n_ville FOREIGN KEY (ville) REFERENCES villes_n(ville)
);
CREATE TABLE produits_n (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(50) NOT NULL,
    categorie VARCHAR(50) NOT NULL
);
CREATE TABLE ventes_n (
    id INT PRIMARY KEY,
    client_id INT NOT NULL,
    produit_id INT NOT NULL,
    montant DECIMAL(10,2) NOT NULL,
    date_vente DATE NOT NULL,
    CONSTRAINT fk_ventes_n_client FOREIGN KEY (client_id) REFERENCES clients_n(id),
    CONSTRAINT fk_ventes_n_produit FOREIGN KEY (produit_id) REFERENCES produits_n(id)
);
```

Les NOT NULL font partie du contrat : sans eux, une vente sans client serait acceptée.

5 : Remplissage (les parents d'abord)

```
INSERT INTO villes_n (ville, province)
SELECT DISTINCT ville_client, province_client FROM ventes_plat;
INSERT INTO clients_n (prenom, ville)
SELECT DISTINCT client, ville_client FROM ventes_plat;
INSERT INTO produits_n (nom, categorie)
SELECT DISTINCT produit, categorie_produit FROM ventes_plat;
INSERT INTO ventes_n (id, client_id, produit_id, montant, date_vente)
SELECT v.id_vente, c.id, p.id, v.montant, v.date_vente
FROM ventes_plat v
INNER JOIN clients_n c ON v.client = c.prenom
INNER JOIN produits_n p ON v.produit = p.nom;
```

Résultat : 2, 4, 6 et 6 lignes. Joindre sur le prénom ne marche que parce que les prénoms sont uniques : il faut une clé fiable.

6 : Reconstitution de la table à plat

```
SELECT vn.id, c.prenom, vi.ville, vi.province, p.nom AS produit, p.categorie, vn.montant, vn.date_
FROM ventes_n vn
INNER JOIN clients_n c ON vn.client_id = c.id
INNER JOIN villes_n vi ON c.ville = vi.ville
INNER JOIN produits_n p ON vn.produit_id = p.id;
```

6 lignes, identiques à ventes_plat.

7 : Retirer la colonne produit (sur la copie)

```
DROP TABLE IF EXISTS commandes_copie;
CREATE TABLE commandes_copie LIKE commandes;
INSERT INTO commandes_copie SELECT * FROM commandes;

ALTER TABLE commandes_copie DROP COLUMN produit;

SELECT o.id, p.nom AS produit, o.montant
FROM commandes_copie o
INNER JOIN produits p ON p.id = o.produit_id;
```

7 commandes, avec le nom du produit venant de produits. La colonne produit dépend de produit_id (dépendance transitive) : elle viole la 3FN. DROP COLUMN est définitif : vérifier d'abord quelles requêtes l'utilisent.

5. Pièges rencontrés

• **Lire l'énoncé mot à mot** : « supérieur ou égal » (>=), 40.00 et non 40.50, 'en préparation' sans faute, NOT NULL demandé, 'Kinshasa-Ville' pour provoquer l'anomalie.

• **Ne pas modifier les vraies données** : un INSERT dans la vraie table clients a transformé la commande orpheline en commande connue. Travailler sur une copie, et copier aussi le parent.

• **Données personnelles** : ne jamais publier de vrai nom ou de vrai numéro dans un dépôt public ; ne pas inventer d'attributs pour un client inconnu.

• **Ordre de création et de suppression** : parents d'abord à la création, enfants d'abord à la suppression.

• **Dépendance transitive** : une colonne qui dépend d'une autre colonne non clé (produit selon produit_id) se déplace dans sa propre table.

• **NULL et clés étrangères** : une valeur NULL n'est jamais comparée à la table parent ; NOT NULL est donc indispensable pour imposer le lien.